Chapter 1

Literature Review

1.1 Traditional Machine Learning for Intrusion Detection

Network intrusion detection has been an active area of research for more than three decades, yet the fundamental tension between detection coverage and operational precision remains unresolved. Early systems relied almost exclusively on rule-based signature matching, an approach exemplified by tools such as Snort [1] and Suricata [2], which compare observed traffic against manually maintained databases of known attack patterns. These systems achieve near-perfect detection for attacks whose signatures are explicitly codified, operate at line rate in many production environments, and are sufficiently interpretable that security analysts can understand precisely why an alert was triggered. However, their fundamental limitation is structural rather than incidental: a signature-based system cannot, by definition, detect what it has not been told to look for. Zero-day exploits, novel variants of known attack families, and polymorphic malware that alters its network-level characteristics to avoid pattern matching all pass through signature-based engines undetected. As the attack surface has expanded and threat actors have grown more sophisticated in evading static rules, this blindness to unknown threats has become the dominant limitation of rule-based intrusion detection.

The response from the research community was to apply machine learning to the intrusion detection problem, framing it as a binary or multiclass classification task over feature vectors derived from network flow records. Random Forests, Support Vector Machines, k-Nearest Neighbour classifiers, and a range of gradient-boosted ensemble methods were applied to successively larger benchmark datasets with considerable apparent success. The progression of evaluation datasets, from KDD CUP 99 through UNSW-NB15 to the CICIDS series produced by the Canadian Institute for Cybersecurity, provided standardised benchmarks that permitted comparison

across studies. On the CICIDS2017 and CSE-CIC-IDS2018 datasets in particular, reported detection accuracies frequently exceeded ninety-nine per cent, leading to the impression that the intrusion detection problem had been largely solved by modern classifiers.

The survey by Ferrag et al. [3] provides a comprehensive comparative assessment of deep learning approaches applied to CSE-CIC-IDS2018, evaluating seven model architectures including convolutional neural networks, recurrent neural networks, deep neural networks, deep belief networks, and deep autoencoders across all fourteen attack categories. Their results demonstrate that the best-performing architectures achieve overall accuracies approaching ninety-seven per cent, with per-attack detection rates varying substantially: near-perfect detection for brute-force attacks but markedly lower rates (around eighty-two per cent) for web-based attacks such as cross-site scripting. The variation in per-attack detection rates highlights a recurring concern in the literature: aggregate accuracy metrics mask significant performance differences across attack types, and classifiers that appear highly effective in overall terms may fail on specific attack categories that are underrepresented in training data or whose network signatures overlap with benign traffic.

Leevy and Khoshgoftaar [4] address this concern directly, examining a substantial corpus of machine learning intrusion detection studies and finding that reported accuracies are suspiciously high across nearly all approaches. Their analysis identifies several methodological factors that inflate benchmark performance: the mixing of training and test flows from the same brief recording window, the absence of temporal holdout splits that would test generalisation to future traffic, the use of evaluation metrics that overweight the dominant benign class, and the inadvertent inclusion of near-duplicate flows that allow classifiers to memorise rather than generalise. The authors conclude that the field's published performance figures systematically overestimate real-world detection capability, and that the true gap between benchmark accuracy and deployment effectiveness is substantially larger than commonly acknowledged. This finding is corroborated by the more recent empirical work of Hesford et al. [5], who evaluated trained classifiers across multiple benchmark datasets (including CICIDS2017 and UNSW-NB15) to assess transferability, and found that precision and recall degraded markedly compared to within-dataset results, with false positive rates that would generate unacceptable alert volumes in operational contexts. Hesford et al. also identify significant reproducibility barriers in the ML-NIDS literature, reporting dependency errors and insufficient documentation that prevented replication of several published results. Hesford et al. [5] attribute this gap to distributional shift: the statistical properties of real network traffic change continuously as applications, user behaviours, and infrastructure evolve, whilst static classifiers trained on historical benchmarks cannot adapt to these drifts without

retraining.

The class imbalance problem presents an additional practical challenge that benchmark studies frequently understate. The CICIDS2018 dataset, which serves as the primary evaluation corpus for this thesis, exhibits an approximately eighty-seven per cent prevalence of benign traffic across all attack categories combined. In individual attack categories, the prevalence of malicious flows is considerably lower. A classifier that predicts every flow as benign achieves roughly eighty-seven per cent accuracy on this dataset whilst detecting precisely zero attacks, a result that is both trivially achieved and entirely useless for security purposes. Techniques for addressing class imbalance, including SMOTE oversampling, undersampling of the majority class, class-weighted loss functions, and threshold adjustment, have been extensively studied [6], but each introduces its own trade-offs. Aggressive oversampling of minority classes can amplify noise in the training set; threshold adjustment optimised on validation data may not transfer to real traffic where attack prevalence differs from the training distribution. The practical implication is that intrusion detection systems must be evaluated at realistic class distributions (matching the attack prevalence of production environments) rather than at balanced or artificially enriched class ratios. This methodological commitment distinguishes the present work from the majority of published LLM-based NIDS studies, which evaluate on small, balanced, or manually curated samples.

Beyond the performance limitations, traditional ML classifiers share a fundamental interpretability deficit. A Random Forest producing a malicious verdict for a network flow does so through an ensemble of hundreds of binary decision trees, the collective output of which is a class probability derived from no single comprehensible reasoning path. Post-hoc explanation methods such as SHAP [7] and LIME [8] can recover approximations of feature importance for individual predictions, but these approximations carry their own limitations: they are local linearisations of a non-linear decision surface, they do not explain how features interact, and they provide no mechanism for contextualising a prediction against the broader threat landscape. The security analyst receiving a SHAP explanation that certain byte-count and packet-ratio features contributed to a malicious classification gains little actionable insight into what type of attack may be occurring, how confident the system is in that assessment, or what the strongest arguments against the classification would be. This interpretability gap, rather than any quantitative performance ceiling, is the central motivation for the LLM-based approach pursued in this thesis.

1.2 Large Language Models in Cybersecurity

The application of large language models to cybersecurity tasks has expanded rapidly following the widespread availability of instruction-following models such as GPT-4, Claude, and open-weight alternatives including LLaMA and Mistral. The foundational concern raised by Gupta et al. [9] in their dual-use framework is that the same capabilities that make LLMs useful defensive tools also make them potentially dangerous offensive instruments. Their analysis, framed through the progression from ChatGPT to what they term ThreatGPT, demonstrates that LLMs can generate functional malware, draft convincing phishing content, identify vulnerabilities in code, and explain attack methodologies in detail sufficient to assist unsophisticated threat actors. This dual-use dynamic has shaped the research community's approach to LLM deployment in cybersecurity, motivating careful evaluation of both the benefits and the risks of integrating generative models into security tooling. For intrusion detection specifically, the dual-use concern is less acute (an intrusion detection system produces verdicts rather than attack capabilities), but the broader question of whether LLMs can be trusted to reason reliably about security-sensitive information remains open.

Feng and Sakurai [10] provide the most comprehensive recent survey of LLM applications in network intrusion detection, tracing the evolution from early prompt-engineering experiments through fine-tuned models and agentic architectures. Their taxonomy distinguishes three waves of LLM-NIDS research: the first wave applied off-the-shelf instruction-following models to raw or lightly processed network flow descriptions and found that zero-shot performance was poor, typically near random; the second wave applied fine-tuning and domain adaptation to overcome the knowledge gap between pre-training corpora (which contain negligible network telemetry) and the target task; and the third wave, which remains nascent, explores agentic and multi-agent configurations where LLMs reason iteratively rather than producing single-pass classifications. The survey identifies a consistent finding across first-wave studies: general-purpose LLMs exhibit substantial uncertainty when reasoning about network features because their pre-training corpora contain little structured telemetry data, and their representations of concepts such as port numbers, byte ratios, and packet inter-arrival times are underspecified relative to the precision required for accurate classification.

The empirical finding of Houssel et al. [11] quantifies this limitation with particular clarity. Their study presented GPT-4 and LLaMA-3 with NetFlow records from standardised intrusion detection datasets, encoding each flow's numerical features as key-value text pairs (e.g., `L4_DST_PORT: 80`), and found that both models achieved approximately fifty per cent macro-averaged precision and recall in zero-shot evalu-

ation, effectively no better than random guessing. Critically, the models' errors were not random: qualitative analysis of individual predictions revealed systematic confusions in both directions, with the models over-classifying certain flows as malicious based on superficial feature patterns (such as atypical port numbers or large file transfers) whilst simultaneously misclassifying genuinely malicious flows as benign when packet lengths and retransmission counts fell within ranges the model associated with normal traffic. Houssel et al. describe their study as an "initial exploration," acknowledging that small sample sizes limit generalisability. They also report severe hallucinations, including fabrication of geographic locations for anonymised IPs, highlighting the risks of deploying LLMs without structured output constraints. This result is significant not because it shows LLMs are useless for intrusion detection (the present work demonstrates otherwise) but because it establishes that direct flow classification is not an appropriate deployment mode for general-purpose LLMs. The model's pre-training does not equip it to adjudicate precisely between feature distributions that differ subtly between benign and malicious traffic; what it can do is reason about the significance of features once their anomalous character has been established through statistical or signature-based analysis. This distinction between LLMs as classifiers and LLMs as explainability agents is a central architectural insight of AMATAS.

The fine-tuning literature reveals a different picture. Gutiérrez-Galeano et al. [12] demonstrate that fine-tuning a T5 encoder-decoder language model on labelled network flow data, transformed into a structured textual representation, yields a detector achieving 99.84 per cent accuracy on the CSE-CIC-IDS2018 dataset, a performance figure competitive with state-of-the-art Random Forest classifiers. This aggregate figure masks significant per-class variation: the model struggled with minority attack categories (SQL Injection, Brute Force-Web, XSS), and the authors note that direct latency comparisons were not feasible due to hardware variations across studies. However, fine-tuned models of this type share the interpretability limitations of traditional classifiers: the model assigns a label through learned parameter weights that encode no meaningful reasoning about why a flow was classified as it was. The high accuracy figure is achieved at the cost of the explainability that motivated the application of language models in the first place. Moreover, fine-tuned models are inherently static (they must be retrained as attack patterns evolve) and they cannot leverage the broad world knowledge that makes prompted LLMs capable of contextualising detections within threat landscapes, mapping findings to MITRE ATT&CK, or reasoning about attack intent from flow-level evidence.

The work of Li et al. [13] on IDS-Agent represents the closest prior work to the AMATAS architecture. IDS-Agent is an LLM-based intrusion detection system that combines prompted reasoning with access to external tools, applying an agentic

loop in which the model iteratively retrieves relevant information before producing a verdict. Evaluated on the ACI-IoT and CIC-IoT datasets, IDS-Agent achieves an F1 score of 0.97 on ACI-IoT and 0.75 on CIC-IoT, a marked improvement over non-agentic LLM baselines and competitive with fine-tuned classifiers on the ACI-IoT benchmark. These results confirm that structuring LLM interaction as an iterative reasoning process rather than a single-pass classification substantially improves performance. However, IDS-Agent employs a single agent with tool access rather than multiple agents with distinct analytical roles, and does not include an adversarial mechanism for false positive control. The present work extends this line of research by distributing analysis across six specialist agents, introducing an adversarial Devil's Advocate component, and evaluating across a broader and more challenging attack taxonomy at realistic class distributions.

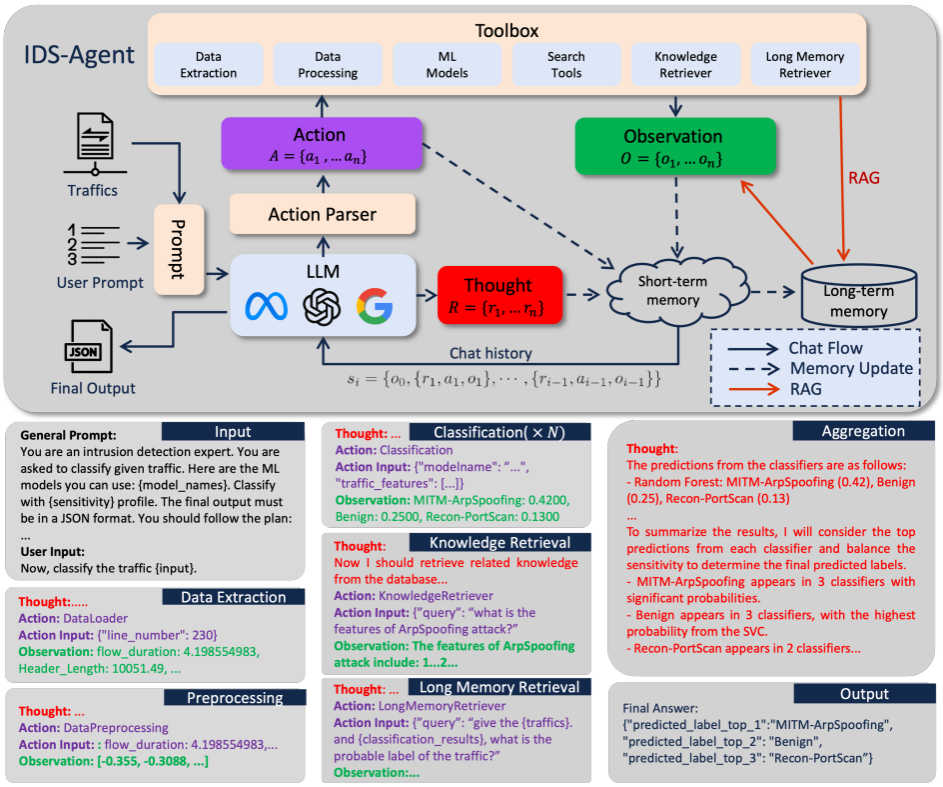

Figure 1.1: The IDS-Agent architecture. A single LLM agent iteratively retrieves information through tool calls (data extraction, ML classification, knowledge retrieval) before producing a verdict. Unlike AMATAS, IDS-Agent employs a single reasoning agent without specialist role diversity or adversarial false positive control. Source: Li et al. [13].

The collective evidence from this literature points towards a nuanced conclusion: LLMs fail as standalone classifiers in the sense explored by Houssel et al. [11], but succeed as reasoning engines when deployed within appropriate architectural

frameworks. The key architectural requirements appear to be that flows are described in natural language that highlights anomalous features rather than presented as raw numerical vectors; that the model is equipped with relevant domain knowledge through system prompting or retrieval; and that reasoning proceeds iteratively or through multiple analytical passes rather than in a single generation. AMATAS addresses all three requirements through its specialist agent design, domain-specific system prompts, and multi-phase consensus architecture.

1.3 Multi-Agent LLM Architectures

The theoretical foundation for multi-agent architectures in artificial intelligence predates large language models by several decades. Minsky [14] proposed that complex intelligent behaviour emerges not from a single unified reasoning process but from the coordinated interaction of many small, specialised agents, each competent within a narrow domain but collectively capable of producing sophisticated, context-sensitive responses. This framework has proved remarkably prescient as a metaphor for multi-agent LLM systems: the observation that diverse specialist perspectives, properly coordinated, outperform any individual agent reasoning in isolation finds strong empirical support in the contemporary literature.

Du et al. [15] provide the first systematic empirical investigation of multi-agent debate as a mechanism for improving LLM reasoning quality. In their framework, multiple language model instances independently generate responses to a question and then iteratively revise their answers based on the responses of other agents, a process of structured argumentation that continues until consensus emerges or a predefined round limit is reached. Evaluated across mathematical reasoning tasks, factual question answering, and strategic reasoning benchmarks, multi-agent debate consistently outperforms single-agent baselines, with gains particularly pronounced on tasks requiring multi-step reasoning where individual agents are prone to early commitment to incorrect intermediate conclusions. The mechanism through which debate improves performance is interpretable: agents correct factual errors surfaced by their debate partners, abandon flawed reasoning chains when challenged, and converge on stronger arguments through structured disagreement. For intrusion detection, where individual agents may over-weight specific features whilst ignoring others, this correction mechanism has direct practical value. Du et al. also caution that debate can converge on confidently wrong answers when all agents share the same initial misconception — a failure mode that motivates the inclusion of a structurally adversarial agent in AMATAS rather than relying on homogeneous debate alone.

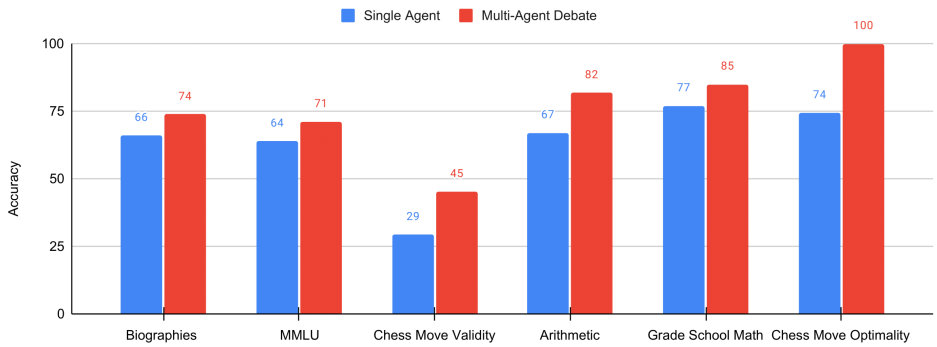

Figure 1.2: Accuracy comparison between single-agent inference and multi-agent debate across six reasoning benchmarks. Multi-agent debate consistently outperforms single-agent baselines, with the largest gains on tasks requiring multi-step reasoning. Source: Du et al. [15].

Question: Carla is downloading a 200 GB file. Normally she can download 2 GB/minute, but 40% of the way through the download, Windows forces a restart to install updates, which takes 20 minutes. Then Carla has to restart the download from the beginning. How load does it take to download the file?

Round 1

ChatGPT: Before the restart, Carla was able to download 40% of the file, which is 0.4 * 200 GB = 80 GB. At a rate of 2 GB/minute, it would take her: time = 80 GB ÷ 2 GB/minute = 40 minutes.

Because of the restart, Carla had to start over. This means she had to download the entire 200 GB file. The time it takes to download the remaining 120 GB is: time = 120 GB ÷ 2 GB/minute = 60 minutes.

But Carla also had to wait 20 minutes for the Windows update to install, so the total time it took to download the file is: total time = 40 minutes + 60 minutes + 20 minutes = **120 minutes**. ✗

Bard: Carla is downloading a 200 GB file at a speed of 2 GB/minute. This means that it will take her 200 / 2 = 100 minutes to download the file.

However, Windows forces a restart to install updates, which takes 20 minutes. This means that Carla will have to restart the download from the beginning. Since Carla has already downloaded 40% of the file, she only needs to download 200 - 40 = 160 GB. At a speed of 2 GB/minute, it will take Carla 160 / 2 = 80 minutes to download the file.

Adding the time for the Windows restart, it will take Carla 80 + 20 = **100 minutes** to download the file. ✗

Round 2

ChatGPT : Based on the calculations provided by the other agent, the correct answer is: Before the restart, Carla was able to download 40% of the file, which is 0.4 * 200 GB = 80 GB. At a rate of 2 GB/minute, it would take her: time = 80 GB ÷ 2 GB/minute = 40 minutes

However, because of the restart, Carla had to start over. This means she had to download the entire 200 GB file. The time it takes to download the entire 200 GB file is: time = 200 GB ÷ 2 GB/minute = 100 minutes

And since Carla had to wait 20 minutes for the Windows update to install, the total time it took to download the file is: total time = 40 minutes + 100 minutes + 20 minutes = **160 minutes**. ✓

Figure 1.3: Example of multi-agent debate between ChatGPT and Bard on a mathematical reasoning task. In Round 1, both agents arrive at incorrect answers through different reasoning errors. In Round 2, ChatGPT corrects its reasoning after reviewing Bard's response, converging on the correct answer of 160 minutes. Source: Du et al. [15].

Chan et al. [16] extend this investigation through the ChatEval framework, which systematically varies the role assignments given to debating agents and measures the effect on response quality across text evaluation tasks. Their central finding is that

diverse role assignments (where each agent is given a distinct analytical persona with different priorities and evaluation criteria) consistently outperform configurations where all agents are assigned identical or similar roles. In the uniform-role condition, agents tend to converge quickly on the first plausible response generated by any participant, as they lack the analytical diversity to surface meaningful objections. In the diverse-role condition, the structural differences in perspective force genuine debate, surfacing weaknesses in preliminary conclusions and producing final outputs of measurably higher quality. This finding directly motivates AMATAS's design decision to assign genuinely distinct analytical mandates to its specialist agents (protocol analysis, statistical anomaly detection, behavioural signature matching, and temporal correlation) rather than deploying multiple instances of a single general-purpose detection agent. Chan et al. additionally found that sequential communication (where agents respond one-by-one with access to prior outputs) was more effective than simultaneous independent generation for ChatGPT agents. AMATAS nonetheless employs parallel specialist execution because the independence constraint ensures no specialist anchors on another's preliminary conclusion, and parallel API dispatch reduces wall-clock latency by a factor of four.

Wu et al. [17] address the practical infrastructure challenge of implementing multi-agent LLM conversations through the AutoGen framework, which provides a conversation management layer enabling arbitrary networks of LLM agents to exchange messages, invoke tools, and produce collaborative outputs. AutoGen's contribution is primarily architectural: it separates the concern of agent coordination from the concern of agent capability, allowing researchers to define multi-agent topologies without re-implementing the messaging and state management infrastructure for each application. The framework has been adopted widely in subsequent research on agentic LLM systems, and its conversation-management primitives informed the design of AMATAS's agent orchestration layer, though AMATAS employs a custom implementation tailored to the specific requirements of flow-level intrusion detection, including cost tracking per agent call, structured output enforcement, and parallel execution of the four specialist phases.

The specific innovation of an adversarial Devil's Advocate agent, a central design element of AMATAS, finds support in recent empirical work examining how adversarial critique improves LLM decision quality. Chiang et al. [18] introduce a Devil's Advocate prompting paradigm in which a secondary LLM agent is instructed to identify flaws in and argue against the primary agent's conclusion, specifically tasked with constructing the strongest possible counter-argument regardless of its own belief about the correct answer. The authors find that this adversarial pressure significantly improves decision quality on complex reasoning tasks, reducing both confident errors (where an agent is wrong but certain) and over-reliance on initial impressions.

Crucially, the improvement in decision quality is asymmetric: the Devil's Advocate intervention is most beneficial when the primary agent's initial conclusion is incorrect, precisely the scenario (a flow that superficially resembles an attack but is genuinely benign) that drives false positive rates in intrusion detection systems. A complementary study by Yang et al. [19] demonstrates that adversarial debate frameworks reduce hallucination rates in LLM outputs by exposing unsupported claims to structured challenge, with agents less likely to assert facts without evidence when they know those assertions will be scrutinised by an adversarial peer. For intrusion detection, where confident but incorrect malicious classifications translate directly into wasted analyst time and degraded trust in the detection system, this hallucination-reduction property of adversarial debate represents a practical operational benefit beyond the accuracy improvements measured in benchmark evaluation.

The collective evidence from multi-agent LLM research supports two architectural commitments that shape AMATAS: first, that specialist role diversity improves collective reasoning quality; and second, that adversarial challenge reduces overconfident errors. Both commitments are implemented structurally in AMATAS rather than through prompt engineering alone: the diversity is enforced by giving each agent a different analytical mandate with distinct evaluation criteria, and the adversarial challenge is enforced by giving the Devil's Advocate an explicit counter-advocacy role independent of the specialists' verdicts.

1.4 Tool-Augmented LLMs and the Model Context Protocol

The reasoning capabilities of large language models can be substantially enhanced by providing access to external tools that retrieve information, perform computations, or interact with external systems. The theoretical basis for tool augmentation as a reasoning enhancement strategy was established by Wei et al. [20] in their introduction of Chain-of-Thought prompting, which demonstrated that instructing LLMs to produce intermediate reasoning steps before final answers markedly improves performance on multi-step tasks. Chain-of-Thought prompting is not tool use in the strict sense (the model reasons using only its parametric knowledge), but it establishes the principle that structured reasoning processes, rather than direct input-to-output mappings, are the appropriate framework for deploying LLMs on complex tasks.

Yao et al. [21] extend this insight through the ReAct framework, which interleaves reasoning traces with actions that invoke external tools, creating a cycle of thought, action, and observation that allows LLMs to gather information dynamically as they

reason. In the ReAct paradigm, an LLM reasoning about whether a network flow is malicious might think through the significance of observed protocol characteristics, decide that it needs to look up the official assignment for a particular port number, execute a tool call to retrieve that information, incorporate the retrieved fact into its ongoing reasoning, and continue the analysis with an enriched knowledge base. This paradigm has been shown to substantially improve performance on knowledge-intensive tasks relative to Chain-of-Thought prompting alone, as tool access allows the model to ground its reasoning in retrieved facts rather than relying on potentially stale or imprecise parametric knowledge. The practical challenge for tool-augmented NIDS is identifying which tools provide genuine value: tools that retrieve information from external threat intelligence databases are only useful if the relevant entities (IP addresses, domain names, malware hashes) are present in those databases, a condition that fails systematically on anonymised or synthetic datasets.

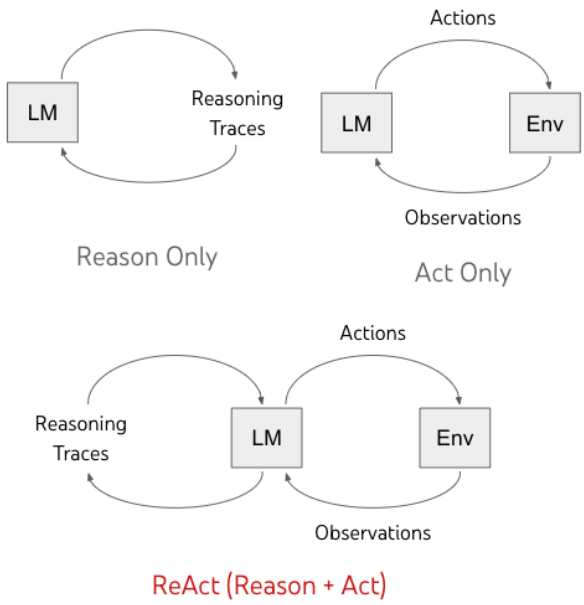

Figure 1.4: The ReAct framework compared to reasoning-only and acting-only paradigms. Reasoning-only (left) generates thought traces without environment interaction; acting-only (centre) takes actions without explicit reasoning. ReAct (bottom) synergises both, interleaving reasoning traces with actions and observations to enable grounded, iterative problem-solving. Source: Yao et al. [21].

Schick et al. [22] formalise tool augmentation through Toolformer, a method for training language models to autonomously decide when and how to call external APIs. Toolformer demonstrates that tool-use capability can be taught through self-supervised learning, with models learning to insert API calls into their generation

process in positions where the retrieved information reduces prediction loss. This work established the viability of tool-augmented LLMs as broadly applicable systems, though Schick et al. note constraints including the inability to chain tools within a single reasoning step, sensitivity to tool description wording, and limited sample efficiency. Toolformer nonetheless demonstrated tool-augmented LLMs as viable beyond narrowly specialised applications, and motivated subsequent research into rich tool ecosystems for LLM-based agents. Lewis et al. [23] provide a complementary contribution through Retrieval-Augmented Generation (RAG), which augments LLM inference with dynamic retrieval from a document corpus, allowing models to access knowledge that was not present at training time or that has changed since the training cutoff. RAG has been applied to cybersecurity tasks including vulnerability assessment, malware analysis, and threat intelligence summarisation, where the need for up-to-date knowledge about recently discovered threats makes static parametric knowledge insufficient.

The Model Context Protocol, introduced by Anthropic [24], provides a standardised interface for connecting LLM agents to external tools, data sources, and services. MCP defines a server-client architecture in which tool providers expose their capabilities through a standardised JSON-RPC protocol, and LLM clients discover available tools, invoke them with structured arguments, and incorporate results into their reasoning context. The significance of MCP for intrusion detection research is that it enables controlled ablation studies: by defining a set of candidate tools and evaluating configurations with and without each tool, researchers can isolate the contribution of specific external information sources to detection performance. The present work employs an MCP-based architecture in its comparison experiments precisely to answer the question of whether external threat intelligence tools (IP reputation databases such as AbuseIPDB, threat feeds from AlienVault OTX, geolocation services) contribute to detection performance on the CICIDS2018 dataset, whose anonymised IP addresses fall in private ranges that do not appear in any external intelligence source. The expected finding, motivated by Qin et al. [25] comprehensive survey of tool learning for language models, is that tool utility is highly dependent on alignment between the tool's knowledge base and the properties of the data being analysed: tools calibrated for real-world public IP addresses provide zero marginal value on traffic from anonymised private ranges.

The MCP comparison experiments conducted in this thesis are designed to make this expected null result rigorous: by holding model capability constant across zero-shot, prompt-engineered, and tool-augmented configurations, and varying only the availability of external tools and the quality of the system prompt, it becomes possible to attribute performance differences unambiguously to those specific interventions. This ablation design, informed by the tool learning survey methodology of Qin et al.

[25], provides a controlled empirical basis for the architectural decision to prioritise multi-agent reasoning over tool augmentation in the AMATAS design.

The trajectory from MCP-centric tool orchestration to multi-agent debate reflects a broader pattern emerging in the LLM security literature: external tools add value only when the data under analysis falls within the tool's coverage domain. For anonymised benchmarks, private address spaces, and synthetic traffic, the tool-augmentation paradigm encounters a hard ceiling that no amount of prompt engineering can overcome. The multi-agent architecture pursued in AMATAS represents an alternative approach to enriching LLM reasoning (through structured disagreement among diverse analytical perspectives rather than through retrieval of external facts), and the comparative evaluation reported in Chapter **??** quantifies the performance gap between these two strategies.

1.5 Explainable AI for Network Security

Explainability in machine learning has become a central concern across application domains as regulations such as the European Union's General Data Protection Regulation and AI Act have introduced requirements for meaningful explanations of automated decisions affecting individuals. In network security, while regulatory requirements are less direct, operational constraints dictate that security analysts must process intrusion detection alerts in real time. The efficacy of this response relies heavily on the system's capacity to provide interpretable classification rationales alongside its verdicts.

The dominant post-hoc explanation methods in the machine learning literature are SHAP (SHapley Additive exPlanations, [7]) and LIME (Local Interpretable Model-Agnostic Explanations, [8]). Both approaches attempt to approximate the behaviour of a complex classifier in the neighbourhood of a specific prediction using a simpler, interpretable surrogate model. SHAP assigns each input feature a contribution value derived from cooperative game theory, quantifying how much that feature shifted the prediction relative to a baseline. LIME fits a local linear model to the classifier's behaviour around the prediction point, using the linear model's coefficients as proxies for feature importance. Both methods have been applied across machine learning domains and are increasingly adopted in security research to interpret classifier decisions. In the intrusion detection context, knowing that packet length variance and backward packet rate were the primary drivers of a malicious classification gives an analyst more to work with than the raw classification probability. Related work on explainability for intrusion detection includes the adversarial perturbation approach of Marino, Wickramasinghe and Manic [26], which identifies the minimum feature modifications needed to correct misclassifications, thereby revealing which features

are most responsible for classification errors.

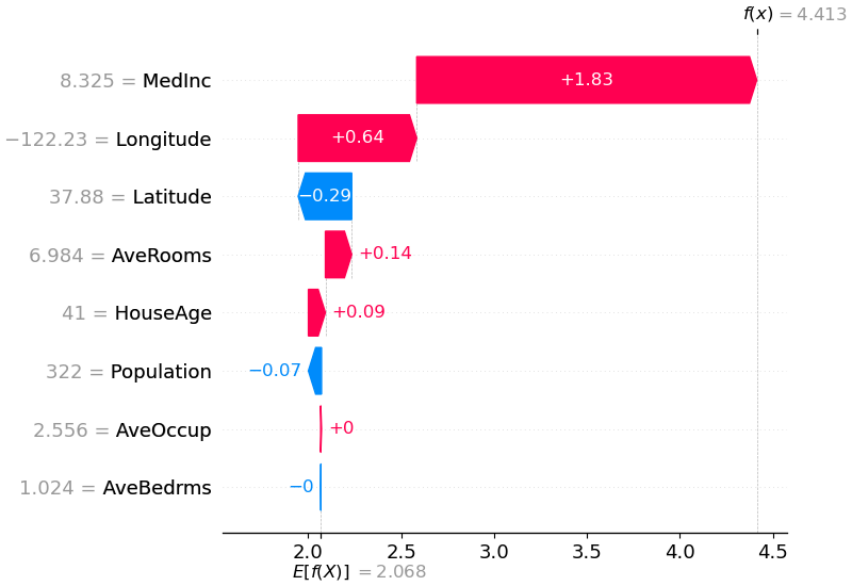

Figure 1.5: A SHAP waterfall plot showing per-feature contributions to a single prediction. Each bar indicates how much a feature's value pushes the prediction above (red) or below (blue) the expected baseline $E[f(X)]$. Whilst informative about *which* features were influential, such explanations do not articulate *what those features mean* in the context of a specific attack scenario or provide counter-arguments. Source: Lundberg and Lee [7]; figure from the SHAP library documentation.

However, these post-hoc methods carry inherent limitations that restrict their utility for security operations. Feature attribution explains which inputs influenced a prediction but not what those inputs mean in the context of a specific attack scenario. A SHAP value indicating that a high byte-per-packet ratio contributed to a malicious classification does not explain whether this pattern is consistent with a DoS flooding attack, a large-file exfiltration, or a legitimate video streaming session. The analyst must supply this contextual reasoning themselves, drawing on domain knowledge that the explanation method cannot provide. Furthermore, SHAP and LIME explain individual predictions in isolation, without contextualising the flagged flow against other flows from the same source, against known attack progressions, or against the temporal pattern of the session. The temporal and behavioural context that human analysts use to distinguish genuine threats from false positives is systematically absent from feature attribution explanations.

LLM-generated reasoning chains provide contextualised explanations that offer distinct operational utility compared to standard statistical feature attribution methods. When AMATAS produces a malicious classification, it accompanies that verdict with natural language reasoning from four specialist agents, each articulating

why specific observed characteristics are anomalous from their analytical perspective, followed by a Devil's Advocate argument for why the flow might be benign, and an Orchestrator synthesis that reconciles all preceding analyses and explains how the final verdict was reached. Instead of outputting raw feature importance weights, the system generates semantic explanations for anomalous features, such as explicitly identifying when a packet rate exceeds the 99th percentile for a given protocol, when a destination port combination deviates from benign service parameters, when two other flows from the same source in the same time window exhibit the same pattern, and when the Devil's Advocate found no compelling benign explanation despite considering common alternative hypotheses. Unlike SHAP, which provides localised mathematical feature attribution, this multi-agent approach outputs natural language rationales intended to improve analyst comprehension and contextualise the threat.

The comparison between feature attribution and LLM reasoning chains has received limited formal attention in the literature, as the deployment of LLMs for intrusion detection explanations is sufficiently recent that systematic user studies have not yet been published. However, the broader XAI literature on human factors in explanation usefulness [27, 28] consistently finds that textual, narrative explanations are more actionable for non-specialist decision-makers than numerical feature importance scores, and that explanations that surface relevant context (why a particular feature value is significant given what is known about the domain) are more effective than those that simply identify which inputs were most influential. These findings, whilst not specific to network security, support the expectation that LLM-generated reasoning chains will provide greater operational value than post-hoc attribution methods, an expectation that the Flow Inspector component of the AMATAS dashboard is designed to facilitate evaluation of.

1.6 Hybrid Architectures and Cost Optimisation

The economic challenge of deploying LLMs for network intrusion detection at scale is not incidental but structural. At current commercial API pricing, processing a single network flow through a six-agent LLM pipeline costs approximately 0.028 US dollars using GPT-4o. The CICIDS2018 dataset contains approximately twenty million flows. Processing all twenty million flows through the full AMATAS pipeline would cost approximately 560,000 US dollars, a figure that renders the approach economically infeasible for any organisation without exceptional security budgets. Even at projected future reductions in inference cost, the order-of-magnitude discrepancy between the cost of LLM analysis and the cost of traditional classifier inference means that pure LLM intrusion detection is not a viable replacement for traditional

approaches; it must be positioned as a complement deployed selectively on the subset of traffic that warrants detailed analysis.

To optimise computational overhead, recent systems have implemented a tiered architecture where a lightweight statistical classifier filters benign traffic, reserving resource-intensive LLM inference exclusively for anomalous flows. Kalafatidis et al. [29] describe a two-tier architecture for software-defined networking and Kubernetes environments in which lightweight statistical anomaly detection models operating on container monitoring data and network flow statistics provide the first detection layer, triggering on-demand LLM-based deep analysis only when anomalies are detected. Their evaluation demonstrates that the statistical pre-filter successfully identifies the majority of benign traffic with high confidence, substantially reducing the load on the LLM analyser without significant reduction in attack recall. This independent discovery of the same two-tier pattern — cheap statistical pre-filtering followed by expensive LLM analysis — provides external validation for the design implemented in AMATAS, and suggests that the approach represents a natural convergence point for researchers working on the cost-accuracy trade-off in LLM-based security systems. The LLM evaluation in Kalafatidis et al. was tested on only 200 packet captures, with minor detection failures on HTTP/2 attacks due to insufficient PCAP sample lengths, indicating a constrained evaluation scope relative to the present work.

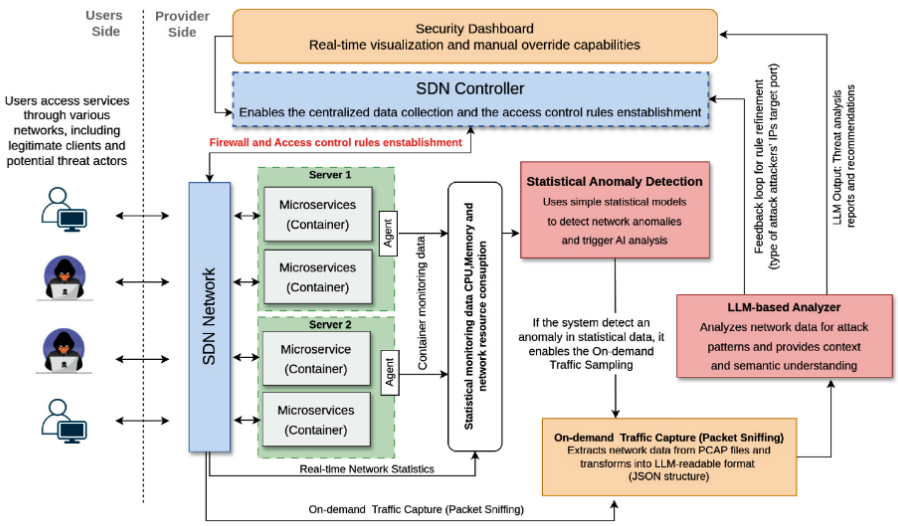

Figure 1.6: The two-tier LLM-enhanced intrusion detection architecture for SDN and Kubernetes environments. Statistical anomaly detection provides continuous monitoring (Tier 1), triggering on-demand LLM-based deep analysis (Tier 2) only when anomalies are detected. This independent design mirrors the AMATAS two-tier approach of cheap ML pre-filtering followed by expensive LLM analysis. Source: Kalafatidis et al. [29].

The choice of operating threshold for the pre-filter presents a fundamental tension between cost and recall. A conservative threshold that allows many uncertain flows through to the LLM tier preserves recall but reduces cost savings; an aggressive threshold that filters more flows reduces cost but risks missing attacks that the Random Forest assessed as borderline benign. AMATAS prioritises recall during the pre-filtering stage by configuring the Tier 1 Random Forest with a maliciousness probability threshold of 0.15. Consequently, any flow exceeding this threshold is forwarded to the LLM pipeline for further analysis. At this threshold, the classifier achieves one hundred per cent recall on attack flows in the development partition (no attack is lost to pre-filtering) whilst correctly filtering approximately ninety-five per cent of benign flows away from the expensive analysis pipeline. The operating threshold was selected empirically on the development dataset rather than on validation or test data, preserving the integrity of the held-out evaluation partitions.

The availability of standardised, large-scale NetFlow datasets has been an important enabling factor for both the machine learning and LLM-based intrusion detection research communities. Sarhan et al. [30] describe the generation and characteristics of standardised NetFlow-based variants of major NIDS datasets, including the NF-CSE-CIC-IDS2018-v2 dataset, which converts the packet-capture data of the original CSE-CIC-IDS2018 dataset into flow records using an extended NetFlow feature set. This conversion is significant because NetFlow is the format in which most enterprise network monitoring infrastructure actually exports traffic data; classifiers and detectors evaluated on NetFlow records have a more direct path to deployment than those evaluated on packet-level features that require full packet capture infrastructure. The CICIDS2018 NetFlow dataset used in this thesis contains approximately twenty million flows covering fourteen attack categories, providing a substantially larger and more diverse evaluation basis than most LLM-NIDS studies, which typically evaluate on hundreds rather than thousands of flows per attack type.

1.7 Research Gap

A synthesis of the preceding review reveals a research landscape with three well-developed areas (traditional ML-based intrusion detection, LLM applications in cybersecurity, and multi-agent LLM architectures) that have not yet been integrated into a coherent system that addresses the full set of challenges facing practical LLM-based NIDS deployment. Each area has made substantial progress within its own boundaries, but the intersections remain underexplored and the combination of techniques required for a practically viable, explainable, cost-efficient multi-agent NIDS has not been previously assembled and evaluated.

The first gap concerns the integration of multi-agent debate with intrusion detection. Du et al. [15], Chan et al. [16], and related work establish that multi-agent debate improves reasoning quality in general LLM tasks, and Li et al. [13] demonstrate that agentic LLM architectures improve intrusion detection performance compared to single-pass classification. However, these threads have not been combined: no prior work applies a structured multi-agent debate architecture (with distinct specialist roles, adversarial challenge, and consensus synthesis) to network intrusion detection. The question of whether the diversity and adversarial properties that make multi-agent debate beneficial in general reasoning tasks translate to the domain-specific, feature-intensive task of intrusion detection remains open.

The second gap concerns false positive control in LLM-based detection. The literature on LLM-NIDS consistently identifies false positive rate as a critical metric; Houssel et al. [11] find that GPT-4 exhibits systematic bias toward certain classifications, and Li et al. [13] note that their single-agent IDS-Agent architecture produces more false positives on the CIC-IoT dataset than on ACI-IoT. No prior work introduces a structural mechanism for false positive control in LLM-based intrusion detection. The present work addresses this gap through the Devil's Advocate agent, whose explicit mandate to construct the strongest possible benign interpretation of any flagged flow provides adversarial pressure against over-classification. The effectiveness of this mechanism, measured by the near-zero false positive rate achieved across the Stage 1 evaluation, constitutes a novel empirical finding with direct practical relevance.

The third gap concerns the integration of ML pre-filtering with multi-agent LLM analysis for cost reduction. The economic infeasibility of pure LLM intrusion detection is well understood but has received little systematic attention in the LLM-NIDS literature, where evaluation datasets are typically small enough that cost is not a binding constraint. The two-tier architecture of Kalafatidis et al. [29] addresses this problem in an SDN context with statistical anomaly detection triggering a single LLM analyser, but does not employ multi-agent debate, specialist role diversity, or adversarial false positive control in the second tier. No prior work presents an empirically validated two-tier architecture that combines an ML pre-filter for cost reduction with a *multi-agent* LLM pipeline for detection and explainability, with cost measured rigorously across realistic class distributions and multiple attack types.

The fourth gap concerns evaluation methodology. The majority of LLM-based NIDS studies evaluate on small samples (typically dozens to a few hundred flows) selected from balanced or near-balanced class distributions that do not reflect the operational reality of network traffic. AMATAS is evaluated on one thousand flows per attack type at a five per cent attack prevalence, matching the realistic class distributions that deployed systems would encounter. Per-attack-type evaluation

at this granularity, spanning all fourteen CICIDS2018 attack categories, enables a systematic mapping of LLM-based detection performance across the attack taxonomy that has not previously been published.

In aggregate, no existing published system combines the four components that together define AMATAS's contribution: a multi-agent LLM debate architecture with specialist role diversity, an adversarial Devil's Advocate mechanism for false positive control, a machine learning pre-filter for cost reduction, and per-attack-type evaluation at realistic class distributions on the CICIDS2018 NetFlow v3 benchmark. The present thesis presents AMATAS as the first system to integrate these components, and the subsequent chapters report the empirical results that establish both the capabilities and the limitations of this integrated architecture.

Bibliography

[1] M. Roesch. 'Snort — Lightweight Intrusion Detection for Networks'. In: *Proceedings of the 13th USENIX Conference on System Administration*. USENIX Association, 1999, pp. 229–238.

[2] Open Information Security Foundation. *Suricata — Open Source IDS/IPS/NSM Engine*. 2010. URL: https://suricata.io (visited on 01/03/2026).

[3] M. A. Ferrag, L. Maglaras, S. Moschoyiannis and H. Janicke. 'Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study'. In: *Journal of Information Security and Applications* 50 (2020), p. 102419. DOI: 10.1016/j.jisa.2019.102419.

[4] J. L. Leevy and T. M. Khoshgoftaar. 'A Survey and Analysis of Intrusion Detection Models Based on CSE-CIC-IDS2018 Big Data'. In: *Journal of Big Data* 7.1 (2020), p. 104. DOI: 10.1186/s40537-020-00382-x.

[5] J. Hesford, D. Cheng, A. Wan, L. Huynh, S. Kim, H. Kim and J. B. Hong. 'Expectations Versus Reality: Evaluating Intrusion Detection Systems in Practice'. In: *Proceedings of the 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks — Supplemental Volume (DSN-S 2025)*. 2025. arXiv: 2403.17458 [cs.CR].

[6] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue and G. Bing. 'Learning from Class-Imbalanced Data: Review of Methods and Applications'. In: *Expert Systems with Applications* 73 (2017), pp. 220–239. DOI: 10.1016/j.eswa.2016.12.035.

[7] S. M. Lundberg and S.-I. Lee. 'A Unified Approach to Interpreting Model Predictions'. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 4765–4774.

[8] M. T. Ribeiro, S. Singh and C. Guestrin. '"Why Should I Trust You?": Explaining the Predictions of Any Classifier'. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1135–1144. DOI: 10.1145/2939672.2939778.

[9] M. Gupta, C. Akiri, K. Aryal, E. Parker and L. Praharaj. 'From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy'. In: *IEEE Access* 11 (2023), pp. 80218–80245. DOI: 10.1109/ACCESS.2023.3300381.

[10] Y. Feng and K. Sakurai. 'Network Intrusion Detection: Evolution from Conventional Approaches to LLM Collaboration and Emerging Risks'. In: *arXiv preprint arXiv:2510.23313* (2025). arXiv: 2510.23313 [cs.CR].

[11] P. R. B. Houssel, P. Singh, S. Layeghy and M. Portmann. 'Towards Explainable Network Intrusion Detection Using Large Language Models'. In: *Proceedings of the 2024 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT 2024)*. 2024, pp. 67–72. DOI: 10.1109/BDCAT63179.2024.00021.

[12] L. Gutiérrez-Galeano, J. Domínguez-Jiménez, J. Schäfer and I. Medina-Bulo. 'LLM-Based Cyberattack Detection Using Network Flow Statistics'. In: *Applied Sciences* 15.12 (2025), p. 6529. DOI: 10.3390/app15126529.

[13] Y. Li, Z. Xiang, N. D. Bastian, D. Song and B. Li. 'IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks'. In: *NeurIPS 2024 Workshop on Open-World Agents*. 2024.

[14] M. Minsky. *The Society of Mind*. New York: Simon and Schuster, 1986.

[15] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum and I. Mordatch. 'Improving Factuality and Reasoning in Language Models through Multiagent Debate'. In: *arXiv preprint arXiv:2305.14325* (2023). arXiv: 2305.14325 [cs.CL].

[16] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu and Z. Liu. 'ChatEval: Towards Better LLM-Based Evaluators through Multi-Agent Debate'. In: *arXiv preprint arXiv:2308.07201* (2023). arXiv: 2308.07201 [cs.CL].

[17] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang and C. Wang. 'AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation'. In: *arXiv preprint arXiv:2308.08155* (2023). arXiv: 2308.08155 [cs.AI].

[18] C.-W. Chiang, Z. Lu, Z. Li and M. Yin. 'Enhancing AI-Assisted Group Decision Making through LLM-Powered Devil's Advocate'. In: *Proceedings of the 29th International Conference on Intelligent User Interfaces (IUI '24)*. 2024, pp. 103–119. DOI: 10.1145/3640543.3645199.

[19] Y. Yang, Y. Ma, H. Feng, Y. Cheng and Z. Han. 'Minimizing Hallucinations and Communication Costs: Adversarial Debate and Voting Mechanisms in LLM-Based Multi-Agents'. In: *Applied Sciences* 15.7 (2025), p. 3676. DOI: 10.3390/app15073676.

[20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le and D. Zhou. 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.

[21] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan and Y. Cao. 'ReAct: Synergizing Reasoning and Acting in Language Models'. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*. 2023.

[22] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda and T. Scialom. 'Toolformer: Language Models Can Teach Themselves to Use Tools'. In: 36 (2023), pp. 68539–68551.

[23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[24] Anthropic. *Model Context Protocol Specification*. 2024. URL: https://modelcontextprotocol.io (visited on 01/03/2026).

[25] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Cong, Y. Zhang, L. Hou, J. Su, M. Sun and B. Zhou. 'Tool Learning with Foundation Models'. In: *ACM Computing Surveys* 56.7 (2024), pp. 1–40. DOI: 10.1145/3656580.

[26] D. L. Marino, C. S. Wickramasinghe and M. Manic. 'An Adversarial Approach for Explainable AI in Intrusion Detection Systems'. In: *Proceedings of the 44th Annual Conference of the IEEE Industrial Electronics Society (IECON 2018)*. 2018, pp. 3225–3231. DOI: 10.1109/IECON.2018.8591457.

[27] I. Lage, E. Chen, J. He, M. Narayanan, B. Kim, S. J. Gershman and F. Doshi-Velez. 'Human Evaluation of Models Built for Interpretability'. In: *Proceedings of the AAAI Workshop on Network Interpretability for Deep Learning*. 2019, pp. 1–5.

[28] M. Chromik and A. Butz. 'Human-XAI Interaction: A Review and Design Principles for Explanation User Interfaces'. In: *Human-Computer Interaction – INTERACT 2021 (IFIP TC 13)*. Vol. 12936. Lecture Notes in Computer Science. Springer, 2021, pp. 619–640. DOI: 10.1007/978-3-030-85616-8_36.

[29] S. Kalafatidis, N. Papageorgopoulos, A. Kartakoullis and G. Ledakis. 'LLM-Enhanced Intrusion Detection for Containerized Applications: A Two-Tier Strategy for SDN and Kubernetes Environments'. In: *Lecture Notes in Computer Science (ARES 2025 EU Projects Symposium Workshops)*. Vol. 15998. Springer, 2025, pp. 55–73. DOI: 10.1007/978-3-032-00642-4_4.

[30] M. Sarhan, S. Layeghy, N. Moustafa and M. Portmann. 'NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems'. In: (2021), pp. 117–135. DOI: 10.1007/978-3-030-86534-4_9.